Member 2 — Dataset + Non-IID Pipeline

Federated learning project documentation (trace-fl)

1. Overview

The goal of Member 2 is to create realistic and reproducible client datasets for federated learning. This module supports:

- MNIST and Fashion-MNIST
- IID partitioning
- Dirichlet-based Non-IID partitioning
- Reproducible client DataLoaders
- Class-distribution visualization

2. Deliverables

Deliverable	Status
MNIST loader	**Complete**
Fashion-MNIST loader	**Complete**
IID partitioning	**Complete**
Dirichlet Non-IID partitioning	**Complete**
α = 1.0, 0.5, 0.1	**Complete**
Reproducible seeds	**Complete**
Client DataLoaders	**Complete**
Class-distribution plots	**Complete**
End-to-end validation	**Complete**

3. Datasets

Dataset	Train	Test	Classes	Image
MNIST	60,000	10,000	10	28 × 28
Fashion-MNIST	60,000	10,000	10	28 × 28

Both datasets are converted to tensors and normalized before local training.

4. Partitioning

IID

IID partitioning randomly divides the training data between clients while keeping the overall class distribution approximately similar. In the experiments, **5 clients** are used, and each client receives **12,000 samples** for both MNIST and Fashion-MNIST.

Dirichlet Non-IID

Dirichlet sampling creates different class distributions across clients. **Smaller α gives stronger class heterogeneity.**

α	Heterogeneity
1.0	Lower
0.5	Higher
0.1	Strong

The partition generator processes each class separately, samples client proportions using a Dirichlet distribution, assigns samples according to those proportions, and finally shuffles each client's indices.

5. Reproducibility

All experiments use **seed = 42**. The implementation uses deterministic random generators for both partitioning and client DataLoader shuffling. The partition tests verify that:

- The same seed produces the same partition
- A different seed produces a different partition
- All samples are assigned exactly once

6. Client DataLoaders

Each client receives a LocalPartition:

```
LocalPartition
├── partition_id
├── train_loader
├── test_loader
└── num_examples
```

The server also has a separate global test DataLoader for centralized evaluation.

7. Class Distribution Visualization

The plotting script `scripts/plot_distributions.py` generates plots for IID, Dirichlet α = 1.0, 0.5 and 0.1, stacked class distributions, and client-size comparison. Output is saved to:

```
artifacts/distributions/
├── mnist/
└── fashion_mnist/
```

8. Experimental Setup

All eight configurations were tested using Flower simulation.

Parameter	Value
Number of clients	5
FL rounds	3
Local epochs	1
Batch size	32
Learning rate	0.01
Random seed	42
Client participation	100%
Device	CPU

9. Experimental Results

Centralized global evaluation accuracy after each federated round. **Round 3 (final)** is in bold. Rows in red are the strongest non-IID setting (α = 0.1).

MNIST

Partition	Round 0	Round 1	Round 2	Round 3
IID	11.16%	90.89%	94.06%	**95.53%**
α = 1.0	10.35%	86.96%	92.32%	**94.37%**
α = 0.5	9.81%	84.43%	93.30%	**95.05%**
α = 0.1	**10.27%**	**43.97%**	**67.46%**	**77.97%**

Fashion-MNIST

Partition	Round 0	Round 1	Round 2	Round 3
IID	12.34%	74.29%	77.93%	**80.25%**
α = 1.0	6.97%	68.66%	76.02%	**79.27%**
α = 0.5	8.05%	58.34%	77.39%	**80.08%**
α = 0.1	**11.60%**	**51.56%**	**56.40%**	**62.09%**

The results show the expected effect of stronger label heterogeneity: **the α = 0.1 setting produced substantially lower global accuracy**, especially during the early rounds.

10. Client Distribution Examples

Dirichlet partitioning produces different client sizes as well as different class distributions. Examples at α = 0.1:

Client	MNIST	Fashion-MNIST
Client 0	8,505	7,681
Client 1	8,904	21,271
Client 2	13,325	8,456
Client 3	21,164	14,212
Client 4	8,102	8,380
Total	**60,000**	**60,000**

11. Class Distribution Plots — MNIST

Heatmaps and bar charts are generated for each partition. Darker cells mean fewer samples; brighter cells mean more.

> **Note:** these plots were generated with 20 clients, whereas the training experiments in Section 8 use 5 clients.

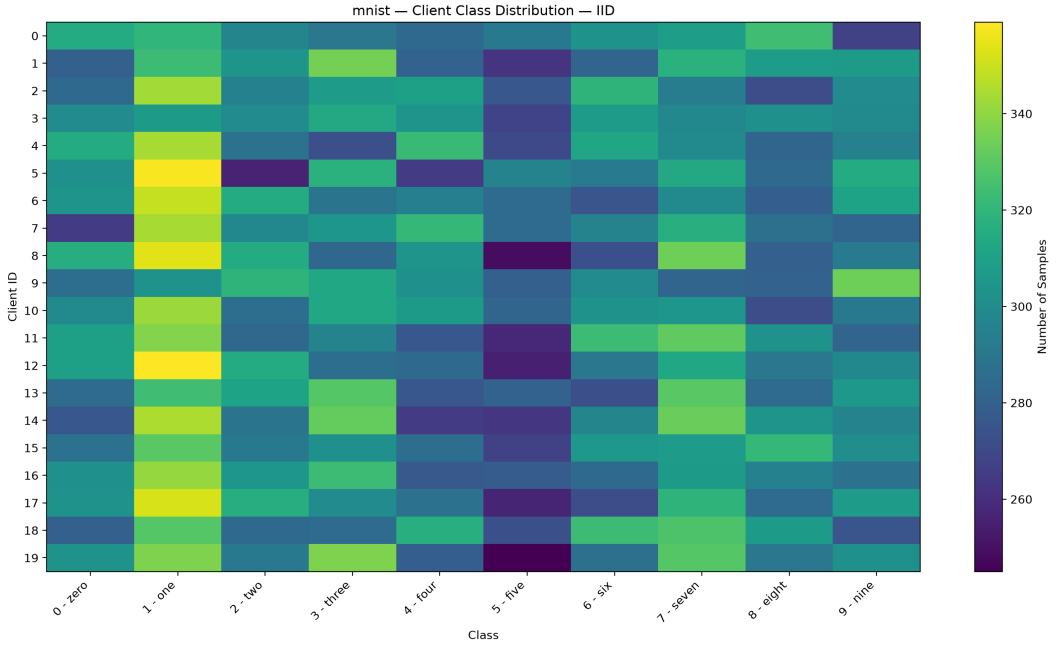


Figure 1. *MNIST — client class distribution, IID (colour range about 250–350 samples per cell).*

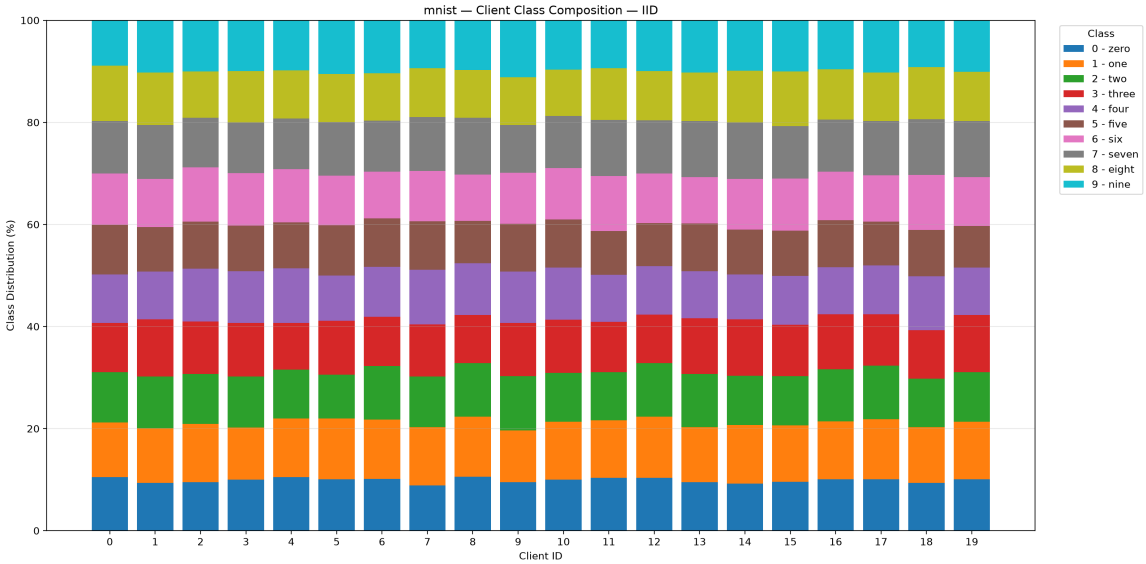


Figure 2. *MNIST — stacked class composition per client, IID (each class is about 10% for every client).*

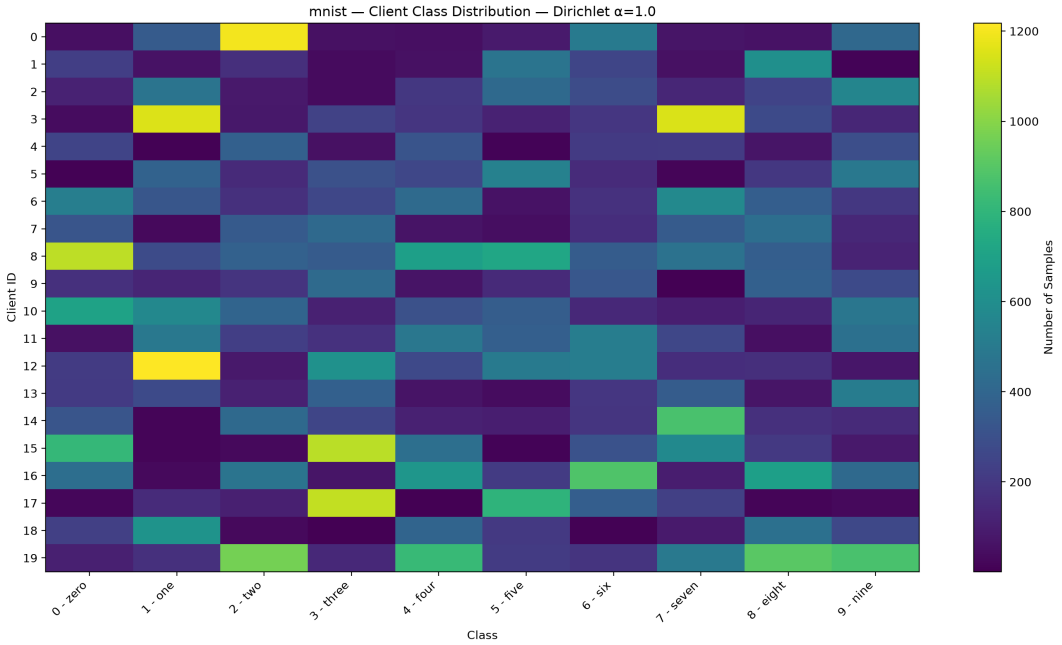


Figure 3. *MNIST — client class distribution, Dirichlet α = 1.0 (**mild skew**).*

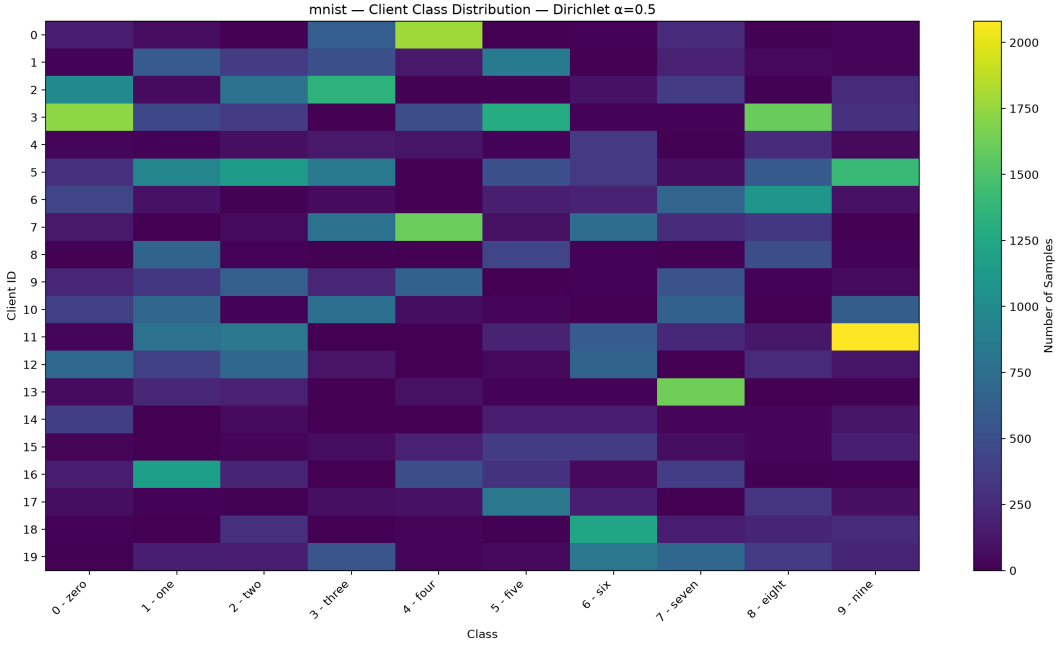


Figure 4. *MNIST — client class distribution, Dirichlet α = 0.5 (**moderate skew**).*

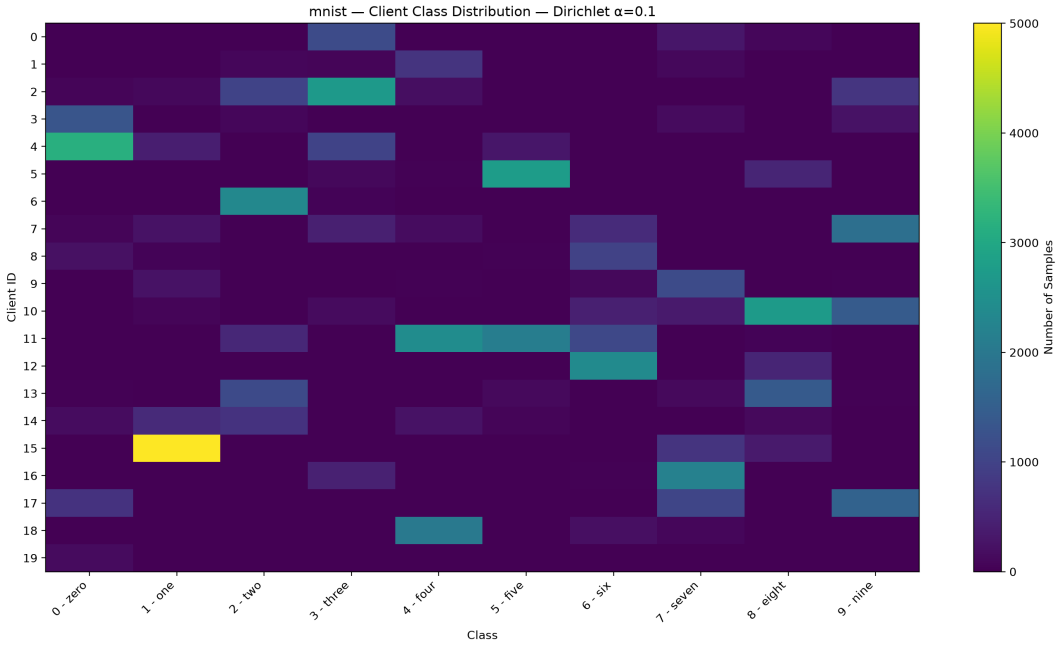


Figure 5. *MNIST — client class distribution, Dirichlet α = 0.1 (**strong skew**: most clients hold only a few classes).*



Figure 6. *MNIST — training samples per client across IID and Dirichlet partitions. IID stays flat; smaller α gives more uneven client sizes.*

12. Class Distribution Plots — Fashion-MNIST

The same plots are generated for Fashion-MNIST. Darker cells mean fewer samples; brighter cells mean more.

> **Note:** these plots were generated with 20 clients, whereas the training experiments in Section 8 use 5 clients.

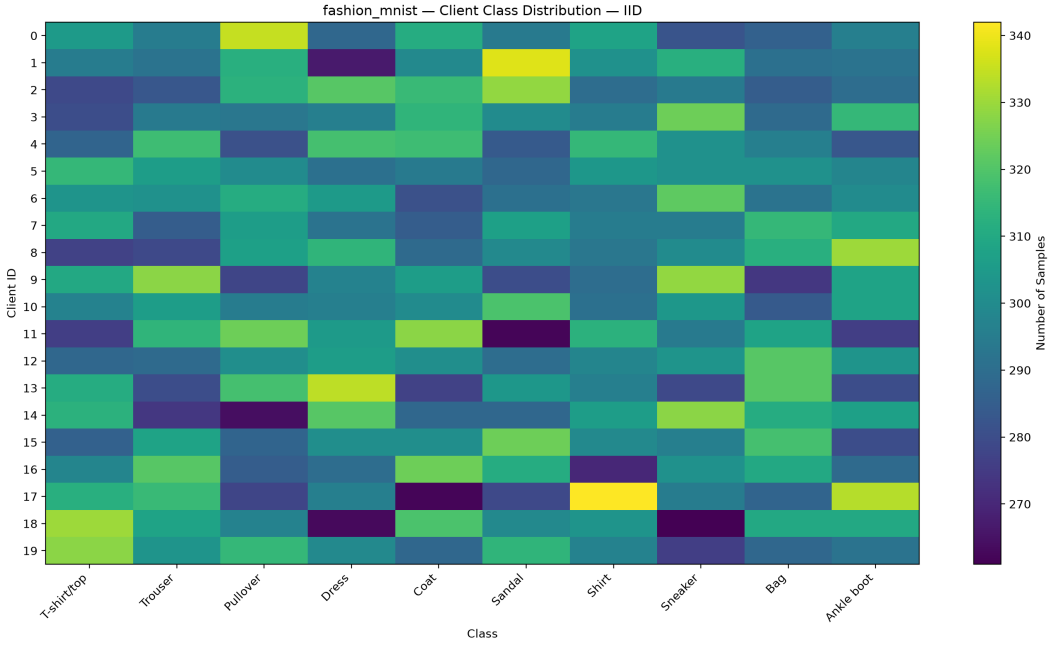


Figure 7. *Fashion-MNIST — client class distribution, IID (colour range about 260–340 samples per cell).*

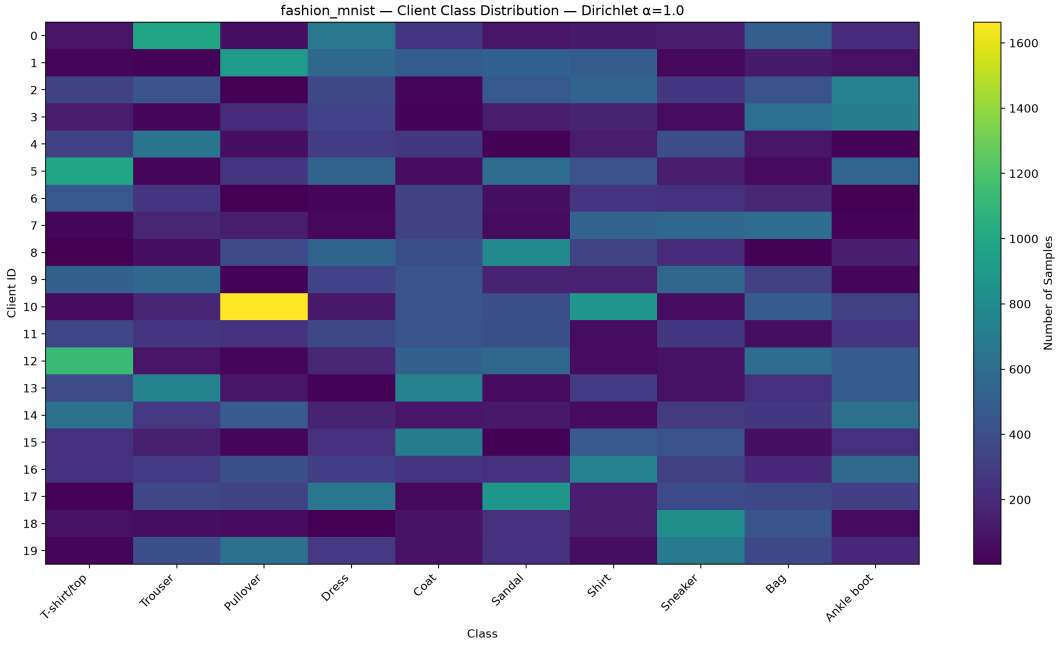


Figure 8. *Fashion-MNIST — client class distribution, Dirichlet α = 1.0 (**mild skew**).*

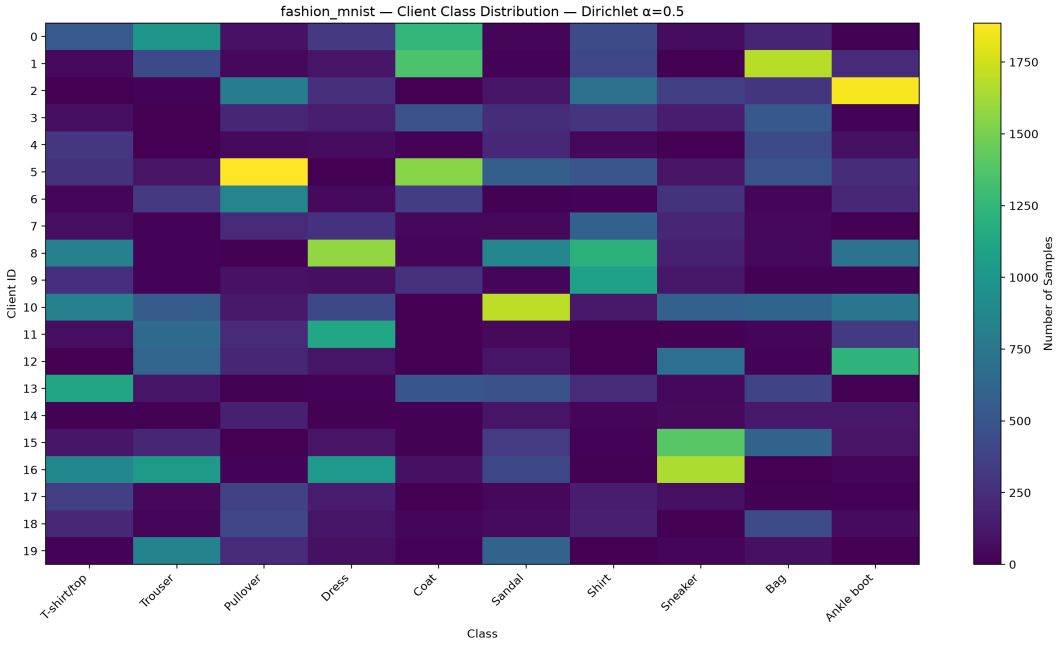


Figure 9. *Fashion-MNIST — client class distribution, Dirichlet α = 0.5 (**moderate skew**).*

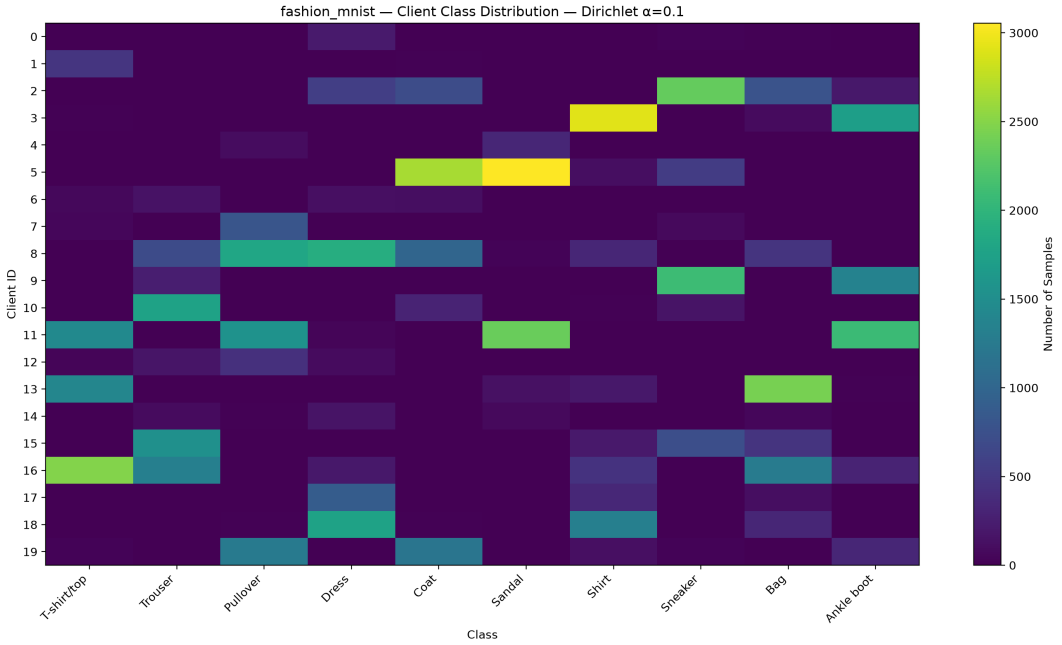


Figure 10. *Fashion-MNIST — client class distribution, Dirichlet α = 0.1 (**strong skew**: most clients hold only a few classes).*



Figure 11. *Fashion-MNIST — training samples per client across IID and Dirichlet partitions. IID stays flat; smaller α gives more uneven client sizes.*

13. Implementation Structure

```
trace-fl/
├── data/
│   ├── dataset.py
│   └── partition.py
├── scripts/
│   └── plot_distributions.py
├── tests/
│   └── test_partition.py
└── artifacts/
    └── distributions/
        ├── mnist/
        └── fashion_mnist/
```

File	Responsibility
`data/dataset.py`	Dataset loading, preprocessing, IID partitioning, Dirichlet partitioning, client DataLoaders, global test DataLoader
`data/partition.py`	`create_dirichlet_partitions()` and `LocalPartition`
`scripts/plot_distributions.py`	Class-distribution and client-size plots

14. Testing

The partitioning tests verify that:

- Every training sample is assigned exactly once
- The same seed produces the same partition
- A different seed produces a different partition
- Invalid partition parameters are rejected

The project test suite result: **15 passed**.

15. Final Validation Matrix

Dataset	IID	α = 1.0	α = 0.5	α = 0.1
MNIST	**Pass**	**Pass**	**Pass**	**Pass**
Fashion-MNIST	**Pass**	**Pass**	**Pass**	**Pass**

Total: 8/8 experiments completed successfully. Each experiment used 5 clients, with all 5 participating in every round and **zero client failures**.

16. Setup and Running

This section lets another team member reproduce every Member 2 result from a clean checkout. Run all commands from the project root (`trace-fl/`).

16.1 Environment and dependencies

The project uses **uv** for environment and dependency management. A single command creates the virtual environment and installs all dependencies:

```
uv sync
```

16.2 Datasets

MNIST and Fashion-MNIST are loaded by `data/dataset.py` and are selected through the `dataset.name` field in the config file (see 16.3). Both are converted to tensors and normalized automatically.

16.3 Configuration: config/config.yaml

All experiment settings live in one file. The example below is the Fashion-MNIST, Dirichlet α = 0.5 run:

```
num_clients: 5
num_rounds: 3
dataset:
  name: fashion_mnist
  partition_type: dirichlet
  alpha: 0.5
local_training:
  epochs: 1
  batch_size: 32
  learning_rate: 0.01
federation:
  fraction_train: 1.0
  min_available_clients: 2
seed: 42
```

16.4 Commands

Task	Command
Install dependencies	`uv sync`
Run the tests	`uv run python -m pytest -q`
Generate distribution plots	`uv run python scripts/plot_distributions.py`
Run a Flower experiment	`uv run flwr run . --stream`

Plots are written to `artifacts/distributions/` (one sub-folder per dataset).

16.5 Reproducing the full experiment matrix

Edit `config/config.yaml` for each run, then execute `uv run flwr run . --stream`. Only the three fields below change between runs; everything else stays as in 16.3 (5 clients, 3 rounds, seed 42). Alpha is only used when the partition type is Dirichlet.

Run	dataset.name	partition_type	alpha	Expected round 3 accuracy
MNIST / IID	mnist	iid	—	**95.53%**
MNIST / α = 1.0	mnist	dirichlet	1.0	**94.37%**
MNIST / α = 0.5	mnist	dirichlet	0.5	**95.05%**
MNIST / α = 0.1	**mnist**	**dirichlet**	**0.1**	**77.97%**
Fashion-MNIST / IID	fashion_mnist	iid	—	**80.25%**
Fashion-MNIST / α = 1.0	fashion_mnist	dirichlet	1.0	**79.27%**
Fashion-MNIST / α = 0.5	fashion_mnist	dirichlet	0.5	**80.08%**
Fashion-MNIST / α = 0.1	**fashion_mnist**	**dirichlet**	**0.1**	**62.09%**

16.6 Expected output

- Tests: **15 passed**.
- Plots: IID, α = 1.0, 0.5, 0.1, stacked and client-size figures saved under `artifacts/distributions/`.
- Flower run: 3 rounds, all 5 clients participating in every round, **zero client failures**.
- Accuracy: Round 0 is near chance level (about 10%). Final accuracies should match the table in 16.5 and Section 9 when the same seed (42) is used; the α = 0.1 runs are clearly the lowest.

17. Conclusion

The Member 2 dataset pipeline is complete. It provides reproducible MNIST and Fashion-MNIST federated datasets with IID and Dirichlet Non-IID partitioning, supporting α = 1.0, 0.5 and 0.1, client DataLoaders, global evaluation, visualization, and partition tests. The full 8/8 experiment matrix was validated through Flower simulation.